

Software Engineering 2 – Zusammenfassung

Vollständige Klausurvorbereitung

Vorlesungsunterlagen SE2, Universität Klagenfurt

Inhaltsverzeichnis

Software Engineering 2 – Zusammenfassung zur Prüfung	1
1. EINFÜHRUNG SOFTWARE ENGINEERING	1
2. SOFTWAREENTWICKLUNGSPROZESSE	2
3. SCRUM	3
4. SOFTWARE DESIGN	6
5. ANFORDERUNGEN (Requirements Engineering)	9
6. QUALITÄTSSICHERUNG	15
7. PROJEKTMANAGEMENT	17
8. KONFIGURATIONSMANAGEMENT	21
ZENTRALE PRÜFUNGSPUNKTE – Quick Reference	24

Software Engineering 2 – Zusammenfassung zur Prüfung

Prüfung: 29.05.2026, 16:30–17:30, HS A, 60 Minuten, online/SPU, ohne Unterlagen, Multiple Choice 60 Punkte – Notenschlüssel: $5 < 36 \leq 4 < 42 \leq 3 < 48 \leq 2 < 54 \leq 1$ Themen: Softwareentwicklungsprozesse, Anforderungen, Software Design, Konfigurationsmanagement, Qualitätssicherung, Projektmanagement

1. EINFÜHRUNG SOFTWARE ENGINEERING

Definition (Sommerville)

Software Engineering ist eine **technische Disziplin**, die sich mit **allen Aspekten** der Software-Produktion beschäftigt – von der frühen Systemspezifikation bis zur Wartung nach der Inbetriebnahme.

- **Technische Disziplin:** Bewusste Selektion von Theorien, Methoden, Werkzeugen unter organisatorischen/-finanziellen Einschränkungen.
- **Alle Aspekte:** Technische Aspekte, Projektmanagement, Entwicklung von Werkzeugen/Methoden zur Unterstützung professioneller Softwareentwicklung.

Unterschied zur Informatik

- **Informatik:** Theorien und Methoden der SW-Systeme.
- **SE:** Probleme der SW-Herstellung – wirtschaftlich, schnell, zuverlässig.

Was ist besonders an Software?

- Unsichtbar, kein physikalischer Platz, keine physikalische Substanz, kein Geruch, kein Aussehen.
- ABER: Risikobehaftet, lebt länger als geplant, kann großen Schaden anrichten (Therac-25, Ariane 5, Heathrow 2018).

4 Merkmale guter Software

1. **Wartbarkeit** (Maintainability)
2. **Sicherheit und Zuverlässigkeit** (Dependability)
3. **Effizienz** (Efficiency)
4. **Benutzerfreundlichkeit / Akzeptanz** (Acceptability)

Herausforderungen

- Heterogenität, neue Technologien, wirtschaftlicher/sozialer Wandel, Sicherheit/Zuverlässigkeit.

„Hacken“ vs. Software Engineering

Personal Software	Industrial-strength Software
Developer is user	Client is user
Bugs are tolerable	Bugs are not tolerated
UI not important	UI important
No/Minor documentation	Lots of documentation
SW not in critical use	Supports business functions
Reliability/Robustness not crucial	Reliability/Robustness is crucial
No investment	Heavy investment (5–25 \$/LOC)
Portability not so important	Portability is an economic advantage

2. SOFTWAREENTWICKLUNGSPROZESSE

Ein **SW-Prozess-Modell** ist eine **abstrakte Repräsentation einer Menge an Aktivitäten und Leistungen**, die zur Erzeugung eines SW-Produkts führen.

Aktivitäten in jedem Prozess

1. **Spezifizierung** (Anforderungen) – Funktionalität und Einschränkungen festlegen.
2. **Design und Implementierung** – Architektur und Entwicklung.
3. **Validierung/Verifizierung** – Erfüllung der Anforderungen / Korrektheit.
4. **Evolution** – Anpassungen / Erweiterungen wegen Änderungen.

Naives Modell: Code & Fix

- Programm schreiben, Fehler finden + beheben.
- (+) Schnell lauffähig, einfaches Testen.
- (–) Nicht planbar, keine Anforderungen, keine Testbasis, teuer in Wartung, schlechte Skalierbarkeit.

Sequenzielle Modelle

Wasserfall (Royce, 1970) **Charakteristika:** – Top-Down, **festgelegte Reihenfolge** der Aktivitäten. – **Eingeschränkte Rückkopplung**, Zwischenprodukt am Ende jeder Phase. – Benutzerbeteiligung **nur zu Beginn**.

(+) Gut planbar (theoretisch), geringer Managementaufwand. (–) Kurskorrekturen nicht frühzeitig erkennbar, Sequenzialität nicht immer nötig, Dokumente werden u. U. wichtiger als das System, Risikofaktoren werden zu spät betrachtet.

V-Modell

- Erweitert Wasserfall um **integrierte Qualitätssicherung** (Verifikation und Validation).
- Sehr umfangreich, für große Projekte entwickelt.
- Phasen, Aktivitäten, Zwischenprodukte.
- **Verifikation:** „Bauen wir das Produkt richtig?“ (gegen Spezifikation).
- **Validation:** „Bauen wir das richtige Produkt?“ (gegen Kundenwünsche).

(+) Detailliert beschrieben (System, QS, Konfigurationsmanagement, PM), generisch, anpassbar, gut für große Projekte, einfach durchzuführen, effizient bei bekannten/konstanten Anforderungen. (–) Software-Bürokratie bei kleinen/mittleren Projekten, Risiken am Schluss („Big Bang“), starr.

Gut anwendbar bei klarer/fixer Funktionalität (OS, DB, Web-Server, Branchen-SW wie SAP R/3).

Iterative Modelle

Spiralmodell (Boehm) **Rundenbasiert** mit 4 Schritten pro Runde: 1. Entwicklungsziele und Einschränkungen 2. **Risikoanalyse** und Alternativen, Prototypen 3. Entwicklungsphase (Design, Programmierung, V&V) 4. Planungsphase für die nächste Iteration

(+) Risiken früher erkannt, volatile Anforderungen besser berücksichtigt, inkrementelle Auslieferung, anpassungsfähig. (–) Mehrarbeit, komplexeres PM, theoretisches Modell.

Agile Prozessmodelle

Agiles Manifest (4 Werte)

1. **Individuen und Interaktionen** über Prozesse und Tools.
2. **Funktionierende Software** über umfassende Dokumentation.
3. **Zusammenarbeit mit dem Kunden** über Vertragsverhandlung.
4. **Reagieren auf Veränderung** über das Befolgen eines Plans.

Charakteristika: - Iterativ (Zyklen: 2 Wochen – einige Monate). - Kleine Teams (6–8 Personen). - Wenig (umfangreiche) Dokumentation. - Kunden sehr wichtig, Präsenz erwünscht. - Lehnt dogmatische Regelungen ab.

(+) Verbessertes Kosten/Nutzen-Verhältnis, bessere Code-Qualität. (–) Ergebnis nicht vorhersagbar, Qualitätseigenschaften nicht garantierbar, weniger Planung, Anforderungen an Kunde, Refactoring-Zeitfaktor.

Agile vs. „Klassische“ Modelle (direkte Vergleichstabelle)

Aspekt	Bisheriger Ansatz	Agiler Ansatz
Ständige Mitwirkung des Kunden	Unwahrscheinlich	Kritischer Erfolgsfaktor
Etwas Nützliches wird geliefert	Erst nach einiger Zeit	Mindestens alle sechs Wochen
Das Richtige entwickeln durch	Langes Spezifizieren, Vorausdenken	Kern entwickeln, zeigen, verbessern
Nötige Disziplin	Formal, wenig	Informell, viel
Änderungen	Erzeugen Widerstand	Werden erwartet und toleriert
Kommunikation	Über Dokumente	Zwischen Menschen
Vorsorge für Änderungen	Durch Versuch der Vorausplanung	Durch „flexibel bleiben“

Es gibt KEIN ideales Modell! Das beste Modell hängt von Organisation, Produkt und Entwickler-Fähigkeiten ab.

3. SCRUM

Begriff

Begriff aus dem Rugby (Nonaka & Takeuchi): “scrum” = Eroberung des Balles. Steht für außergewöhnlich erfolgreiche, **kleine, selbst-organisierte** Produktentwicklungsteams mit vorgegebener Richtung (Produkt) und frei wählbarer Taktik.

SCRUM in 100 Worten

- **Agiler Prozess** mit Fokus auf Auslieferung der wichtigsten Geschäftsanforderungen innerhalb kürzester Zeit.
- Regelmäßige Auslieferung von **tatsächlich lauffähiger Software** (2 Wochen – 1 Monat).
- **Business** setzt Prioritäten, **selbstorganisierende** Entwicklungsteams legen Vorgehen fest.

Prinzipien

- Transparenz
- Beobachtung und Anpassung
- Time-Boxing
- Dinge abschließen
- Maximierung vom Geschäftswert
- Teams scheitern nicht

Rollen

Team (Entwicklungsteam)

- **5–10 Vollzeit-Mitglieder.**
- Heterogen (QA, Dev, Tester, UI Designer), **aber alle machen alles** (fast notwendig).

- Verantwortlich für **Sprint-Erfolg (Commitment)**.
- **Selbstorganisierend**, idealerweise keine Titel.

Scrum Master

- Repräsentiert das Management gegenüber dem Projekt.
- Verantwortlich für **Einhaltung der Scrum-Werte und -Techniken**.
- Stellt sicher, dass Team vollständig, funktional, produktiv ist.
- Unterstützt Zusammenarbeit, **schützt das Team vor äußeren Störungen, entfernt Hindernisse**.

Product Owner

- Repräsentiert den **Kunden**.
- **Definition von Features**, Auslieferungsdatum und Inhalt.
- Verantwortlich für **ROI** (Return on Investment).
- **Priorisierung** der Features, Anpassung nach Bedarf.
- Akzeptanz/Zurückweisung der Arbeitsergebnisse.

Ablauf

Product Backlog

- Liste **aller erforderlichen Projektarbeiten** (story-based, task-based).
- **Vom Product Owner priorisiert**.
- Je höher priorisiert, desto **genauer** spezifiziert.
- Wird **laufend ergänzt und verfeinert**.
- Jedes Backlog Item ist eine **User Story**.

Kano-Modell (Kundenzufriedenheit) 5 Qualitätsebenen: 1. **Basis-Merkmale** (z. B. Sicherheit, Rostschutz) – unterbewusstes Wissen. 2. **Begeisterungs-Merkmale** (Sonderausstattung, Design) – unbewusstes Wissen. 3. **Leistungs-Merkmale** (Beschleunigung, Verbrauch) – bewusstes Wissen. 4. **Unerhebliche Merkmale** (Farbe, Schiebedach). 5. **Rückweisungs-Merkmale** (Dellen, keine Zulassung).

Gewöhnungseffekt: Begeisterungs-Merkmal → Basis-Merkmal (Beispiel: SMS).

User Stories Format: „Als will ich [, so dass ich].“ Beispiel: „Als Vertriebsmitarbeiter will ich nach Tweets regional gruppieren, so dass ich Hotspots finde.“

Wert für Kunde / Product Owner, keine technischen Details (Kundensprache), iterativ verfeinert, gute Planungsgröße, Weg vom Schreiben zum Sprechen, **Akzeptanzkriterien**.

INVEST-Kriterien für User Stories

- **Independent:** unabhängig voneinander
- **Negotiable:** verhandelbar, offen für Veränderung (aber **nicht während eines Sprints**)
- **Valuable:** Mehrwert für Anwender
- **Estimateable:** relative Größe schätzbar
- **Small:** konkret (z. B. „Benutzer soll sich anmelden können“ statt „Benutzerverwaltung“)
- **Testable:** testbar, mit Akzeptanzkriterien

Definition of Done (DoD)

- Wann ist etwas „fertig“?
- Sammlung an Bedingungen, sehr genau, **messbar**, team-/projektspezifisch.
- Bei fehlender / missachteter DoD ist der **Scrum Master** zuständig.

Sprint Backlog

- **Annahme:** Team schafft z. B. 50 Story Points/Sprint (= **Velocity**).
- Welche Stories kommen in den Sprint? → entscheidet **das Team**.
- Manager weisen keine Arbeit zu, entscheiden nichts für das Team.
- Sprint-Backlog kann **nur vom Team** geändert werden.
- Tasks werden vom Backlog abgeleitet.

Sprint

- Dauer: ca. **1 Monat (+/- 1–2 Wochen)**, konstante Dauer für Rhythmus.
- Produkt wird in einem Sprint **entworfen, kodiert UND getestet** (überlappende Entwicklung).
- **Keine Änderungen während des Sprints** – sonst Sprint-Ende.
- **Keine Einschränkungen** für das Team (Scrum Master verantwortlich).

Produktionssupport

- Meistens **nicht planbar**.
- Zwei Kategorien:
 1. Einschreiten **sofort notwendig** (Produktion steht) → **Sprintende** und Reparatur.
 2. Einschreiten **kann geplant werden** (Fehler vorhanden, aber Produktion läuft).

Meetings/Zeremonien

Sprint Planning

- Input: Product Backlog, Team Capabilities, Business Conditions, Technology, Current Product.
- Teilnehmer: Product Owner, Team, Scrum Master, Customers, Management.
- Output: **Sprint Goal** und **Sprint Backlog**.

Daily Scrum Meeting

- **Täglich, 15 Minuten, Stand-up**.
- **Nicht zur Problemlösung** (das danach).
- Drei Fragen:
 1. Was hast du getan?
 2. Was wirst du tun?
 3. (Was ist dazwischengekommen?)
- **„Hühner und Schweine“ nehmen teil – nur Schweine reden** (haben das Commitment).
- Warum täglich? “Ein Projekt verspätet sich um ein Jahr – um einen Tag zu jedem Zeitpunkt” (Fred Brooks, *The Mythical Man-Month*).
- **Kann NICHT durch E-Mail ersetzt werden** – das ganze Team bekommt täglich einen Überblick, Peer-Pressure.

Sprint Review

- Team präsentiert, was es während des Sprints erreicht hat (Demo).
- **Informell** („2-Stunden-Vorbereitung“-Regel).
- Teilnehmer: Kunden, Management, Product Owner, Entwickler.

Sprint Retrospektive

- **Nur das Scrum Team**.
- Feedback-Meeting, Meta-Fragen, Reflexion:
 - Was können wir besser machen? (start)
 - Was hat gut funktioniert? (continue)
 - Was weniger gut? (stop)
- **Keinesfalls** Leistungs-/Performance-Review oder Bewertung!
- **Nicht auslassen** für die ersten 5–6 Sprints.

Artefakte

Burndown Chart

- Zeigt verbleibenden **Gesamtaufwand** des Teams pro Tag.
- Täglich aktualisiert, neu geschätzt.
- Ziel: Team selbst organisieren, Prioritäten setzen.
- So einfach wie möglich, **kein Selbstzweck**.
- Falls missbraucht (Mgmt Review, Ampel-Warnung,...) → **abschaffen**.

Scrum Board

- „Todo-Liste“ des Teams.
- Alle Tasks des aktuellen Sprints.
- Spalten = „Stationen“ (z. B. In Arbeit, Test).
- Offline oder online.

Agiles Schätzen

Was wird geschätzt? → Aufwand pro User Story (im Vergleich!). **Wer schätzt?** → Das Team. **Wann wird geschätzt?** → Vor dem 1. Sprint (theoretisch); 1–3 Sprints im voraus (praktisch). **Wie wird geschätzt?** → Story Points + Planning Poker.

- Schätzungen sind **keine Verpflichtungen**.
- Personentag != Arbeitstag (Overhead).
- **Vergleiche schätzen**, nicht Dauer.
- Größenordnungen: 1, 2, 3, 5, 8, 13, 20, 40, 100, ... (Fibonacci-artig).
- **> 13 Story Points = Epic** → kann **nicht in einem Sprint** umgesetzt werden, muss vor Implementierung **geteilt** werden.

Planning Poker

- Time-Boxed: 15 Minuten pro Story.
- Keine perfekte Schätzung, **Referenz-Story**, Triangularisierung nach mehreren Stories.
- Kriterien: Komplexität, Risiken, Kenntnis, externe Abhängigkeiten, DB-Tabellen.
- Vorgehen: Story vorstellen → Diskussion → Schätzen → Argumentation größter/kleinsten Schätzung → wiederholen.
- **Zu große Differenz?** → Anforderung nicht zu 100 % verstanden, ggf. Klärungsbedarf mit Kunden.

Velocity

- **Entwicklungsgeschwindigkeit des Teams** (Story Points/Sprint).
- Bei Urlaub/Krankheit kann Velocity nicht ausgeschöpft werden.
- **Nur 100 % fertige User Stories zählen** (80 % = 0 Mehrwert für Kunde).
- Berechnung:
 - 1. Sprint: Bauchgefühl.
 - Folgende: „Übernommene Velocity“ (= fertige Story Points letzter Sprint) oder **Median der letzten x Sprints**.
- Variiert über Zeit (Krankheit, Urlaub, schlechte Tage).

Funktionale vs. Nicht-Funktionale Anforderungen

- **Funktional:** Features. „Als Admin möchte ich User löschen können.“
- **Nicht-Funktional:** Performance, Security. „Als Spieler möchte ich, dass das Erstellen des Spielbretts nicht länger als 2 Sekunden dauert.“

Weitere agile Methoden

- **eXtreme Programming (XP):** Werte: Kommunikation, Feedback, Einfachheit, Mut. Rollen: Programmierer, Kunde, XP-Trainer, Verfolger, Tester, Berater, Big Boss.
- **Kanban:** Strategie der Supermarkt-Regale, Kanban-Board zur Visualisierung, Kanban-Karten pro Aufgabe.

4. SOFTWARE DESIGN

Schichtenarchitektur (hierarchisches Design)

- Schichten **kapseln Funktionalität** (Services).
- Subsysteme beliebiger Komplexität, **Interfaces**.
- **Strikte Aufrufhierarchie** von oben nach unten.
- Obere Schichten: **Darstellung (UI)** und Verarbeitung (**Businesslogik**).
- Untere Schichten: **technische Aspekte** (Hardware, DB).

Typische Schichten: - **Presentation Layer:** (graphisches) UI. - **Application Layer** (optional): Koordination und Delegation. - **Business Layer:** Constraints, Validierung, Regeln. - **Technical Layer:** Persistenz, Hardware-Interfaces.

(+) Schichten unabhängig, austauschbar (Interfaces!), reduziert Abhängigkeiten, leicht verständlich. (-) Performance (Anfragen müssen durch alle Schichten – Lösung: **Layer Bridge**), Änderungen in allen Schichten, schlechte Skalierbarkeit.

MVC (Model-View-Controller) – interaktionsorientiertes Design

- **Einsatzbereich:** Presentation Layer (UI).
- **Ziel:** Daten von Anzeige/View loslösen.
- **Model:** enthält Daten/Status, benachrichtigt View bei Update, Anwendungslogik.
- **View:** Anzeige der Daten (HTML, PDF, CSV), leitet User-Eingaben weiter.
- **Controller:** interpretiert/validiert User-Eingaben, triggert Model Update.

(+) Klare Trennung, mehrere Views zu einem Model, leicht änderbar, viele Frameworks. (-) Zusätzliche Abstraktionsschicht, aufwändiges Debugging, Performance bei vielen View/Controller-Paaren.

MVP (Model-View-Presenter)

- **Ableitung von MVC.**
- **Einsatzbereich:** Presentation Layer.
- **Ziel:** Anzeige von Logik loslösen.
- **Model:** Daten/Status, Anwendungslogik.
- **(passive) View:** Anzeige, leitet Input an Presenter weiter.
- **Presenter:** Mittelsmann zwischen Model und View, triggert Business-Logik und View-Updates.
- **Erhöhte Testbarkeit** im Vergleich zu MVC.

Component-Based Software Engineering (CBSE)

Reuse (Wiederverwendung) “Bei jeder SW-Entwicklung wird in der Regel das Rad mehrfach neu erfunden” (Balzert).

(+) **Vorteile:** Produktivität, Beschleunigung, Kostenreduktion, Qualität, Fokus aufs Kerngeschäft, Standards. (-) **Nachteile:** NIH-Syndrom („Not Invented Here”), Kosten für Suchen/Verstehen/Anpassen, Langzeit-Support unsicher. ABER: Software kann wiederverwendet werden, **sie muss es nur**.

Vom Objekt zum Service

1. **Direkter Aufruf:** `new WordImport()` – einfach, IDE-Support, aber **schwer austauschbar**, keine Laufzeit-Konfiguration.
2. **Aufruf via Interface:** `private TextImport ti = new HtmlImport();` – Code muss nicht angepasst werden, aber **Code muss kompiliert werden bei Wechsel**.
3. **Komponente:** klare/stabile Schnittstelle, höhere Abstraktion, wiederverwendbar, locker verbunden.
4. **Service:** zustandslos, plattformunabhängig, klar definierte Schnittstelle, Beschreibung der Schnittstelle plattformunabhängig, **angeboten über Netzwerk**.

Software Komponente – Eigenschaften

- Klare und **stabile Schnittstelle**.
- Höhere Abstraktion.
- Auf Wiederverwendbarkeit ausgelegt.
- **Locker verbunden**.

Definitionen: - *Szyperski*: Eine Einheit der Komposition mit vertraglich spezifizierten Schnittstellen und expliziten Kontextabhängigkeiten. Kann **unabhängig deployed** werden, unterliegt der Komposition durch Dritte. - 4 Eigenschaften: **standardisiert/dokumentiert, unabhängig, zusammensetzbar, verteilbar**.

Komponenten-Framework

- Kümmt sich um **Verdrahtung** der Komponenten.
- **Konfiguration**.
- **Lebenszyklus**.
- Meist auf eine Plattform beschränkt.

Framework

- Reusable Design, das Software-Komponenten benötigt.
- Bestandteile: Framework Core, Framework Library (vom Framework verwendet), Application (vom Anwender entwickelt), Application Extensions (Erweiterung der Framework-Klassen, Bsp.: `extends AppCompatActivity`).

(+) Wiederverwendbarkeit, Modularität (vorgegebene Architektur), Erweiterbarkeit, Standard-Lösungen. (–) Komplexität, Abhängigkeiten, Einarbeitung, Fehlerfindung im Framework.

Auflösung der Abhängigkeiten

- **Konventionell**: Komponente regelt Auflösung, Initialisierung, Kommunikation (direkt oder über Factory).
- **Inversion of Control (IoC)**: Client definiert Abhängigkeiten, **Framework regelt Auflösung**. → Entkopplung.

Dependency Injection (DI)

- **Pattern** (Umsetzung von IoC).
- Client kennt nur die **öffentliche Schnittstelle**.
- Konfiguration der konkreten Implementierung an zentraler Stelle (XML) oder im Code (Annotations, **Auto-wiring**).
- Framework lädt Konfiguration **zur Laufzeit** und injiziert konkrete Implementierung.
- Varianten: **Setter Injection, Constructor Injection**.
- Beans im Container typischerweise als **Singleton** behandelt.

(+) Strukturierung, Abhängigkeiten auf Schnittstellen reduziert, **bessere Testbarkeit** (auch Mocking), Wiederverwendbarkeit. (–) Komplexität, Einarbeitung.

Aspektorientierte Programmierung (AOP)

- Auslagerung von **Cross Cutting Concerns** (Probleme, die in vielen Methoden/Schichten ähnlich auftreten) in **Aspekte**.
- Typische Beispiele: **Logging, Security, Transaktionssteuerung**.

Begriffe: - **Aspekt**: Klasse mit Querschnittsfunktionalität (enthält Pointcuts und Advices). - **Advice**: Implementierung des Aspekts, eingebunden an Join Points (Before, After, Around Advice). - **Join Point**: Punkt, an dem Aspekte eingeführt werden können (Methodenaufruf, -ende, Exception, Variablen-Zugriff, Klassen-/Objekt-Initialisierung). - **Pointcut**: Gruppe von Join Points mit bestimmten Eigenschaften (Muster). - **Introduction**: Einbindung von Attributen/Operationen statisch beim Übersetzen.

Services & SOA

Anforderungen an Services

- **Zustandslos**, lose Kopplung, austauschbar.
- Service Registry, Verteilung, Geschlossenheit.
- Ortstransparenz, Plattformunabhängigkeit, Zugriff über Schnittstelle.

SOA (Service Oriented Architecture)

- Kapselt Verarbeitungsfunktionen **als Dienste** über das Netzwerk, **standardisierte Schnittstellen**.
- Unabhängig von Plattform/Technologie, **höherer Wiederverwendungsgrad** als Komponenten.

Ablauf: 1. Serviceanbieter veröffentlicht im Serviceverzeichnis. 2. Servicenutzer macht Suchanfrage. 3. Verweis auf Service. 4. Serviceaufruf. 5. Serviceantwort.

Realisierungen von SOA

- **XML-basierte Web Services**
- **RESTful Web Services**
- **Microservices**
- **CORBA** (Common Object Request Broker Architecture)

XML-basierte Web Services

- **UDDI** (Universal Description, Discovery, Integration): Standard für Datenaustausch mit Serviceverzeichnis.
- **WSDL** (Web Service Description Language): Beschreibung von Services. Hauptelemente: types, message, portType, binding.
- **SOAP** (Simple Object Access Protocol): Serviceanfrage/-aufruf. Aufbau: **Envelope, (Header), Body, (Fault)**.

RESTful Web Services

- **Representational State Transfer.**
- Ressourcen über HTTP, identifiziert via **URLs/global IDs**.
- Datenformate: JSON, XML, CSV, HTML.
- HTTP-Requests:
 - **GET**: Daten abfragen
 - **POST**: Daten erstellen
 - **PUT**: Update/Ersetzung
 - **DELETE**: Löschen

REST vs. XML-basiert

RESTful	XML-basiert
(+) einfacher zu verwenden	(-) höherer Implementierungsaufwand
(+) Modifikation nur bei URL	(-) starke Bindung Client-Service
(+) JSON, CSV, XML	(-) nur XML
(-) keine standardisierte Beschreibungssprache	(+) WSDL als Beschreibungssprache
(-) kein Verzeichnis	(+) UDDI als Verzeichnis

Microservices

- Applikation aus **Sammlung kleiner Services**.
- Neue Features/Änderungen/Bugfixes können rasch ausgeliefert werden.
- Eigenschaften: lose gekoppelt, leicht wartbar, einzeln testbar, unabhängig zusammensetzbar, austauschbar.
- Bsp.: Amazon Microservices.

SOA gesamt (+) Reusability, Maintainability, Ortsunabhängigkeit, Plattformunabhängigkeit, Technologieunabhängigkeit. (-) Overhead, Service Management, externe Services = Black Box.

Reuse-Ansätze (Übersicht nach Sommerville)

- **Design Patterns** – generische Abstraktion über Anwendungsgrenzen.
- **Component-based Development** – Komponenten gemäß Standard.
- **Application Frameworks** – Klassensammlungen adaptieren/erweitern.
- **Legacy Systems Wrapping** – Schnittstellen über Altsystem.
- **Service-oriented Systems** – Services verlinken.
- **Application Product Lines** – generalisierte Anwendung anpassen.
- **COTS Integration** (Commercial Off-The-Shelf).
- **Configurable Vertical Applications.**
- **Program Libraries** – häufig verwendete Klassen.
- **Program Generators.**
- **Aspect-oriented SW Development.**

5. ANFORDERUNGEN (Requirements Engineering)

Motivation

Top-Gründe für Projekt-Abbruch (Chaos Report): - Unvollständige Anforderungen (13,1 %) - Kunden nicht ausreichend einbezogen (12,4 %) - Mittel nicht ausreichend (10,6 %) - Unrealistische Erwartungen (9,9 %) - Mangelnde Unterstützung Management (9,3 %) - Änderungen in Anforderungen (8,7 %) - Mangelnde Planung (8,7 %)

Definitionen (IREB Glossary)

Requirements Engineering: Ein systematischer und disziplinierter Ansatz für die Spezifikation und das Management von Anforderungen, mit den Zielen: 1. Anforderungen kennen, Konsens unter Stakeholdern erreichen, dokumentieren, systematisch verwalten. 2. Wünsche und Bedürfnisse der Stakeholder verstehen/dokumentieren. 3. Anforderungen so spezifizieren/managen, dass das Risiko minimiert wird, ein System auszuliefern, das die Wünsche der Stakeholder nicht erfüllt.

Stakeholder: Person oder Organisation mit (direktem oder indirektem) Einfluss auf die Anforderungen eines Systems. Beispiele: Anwender, Management, Kunde, Servicepersonal, Product Owner, Kundendienst, Marketing/Vertrieb, Schulungspersonal.

Requirement (Anforderung) [IREB]: 1. Ein Bedürfnis eines Stakeholders. 2. Eine Fähigkeit oder Eigenschaft, die das System haben soll. 3. Eine dokumentierte Repräsentation davon.

Aufgaben des RE (Merken!): Ermitteln, Dokumentieren, Prüfen, Abstimmen und Verwalten von Anforderungen.

Aktivitäten des RE (Sommerville)

1. **Durchführbarkeitsstudie:** Leistet das System einen Beitrag zu Unternehmenszielen? Realisierbar in Zeit/-Kosten/Technik? Mit existierenden Systemen interoperabel?
2. **Anforderungsbestimmung und Analyse:** Verstehen, Sammeln, Klassifizieren, Konflikte lösen, Prioritäten.
3. **Anforderungsspezifikation.**
4. **Anforderungvalidierung:** Konsistenz, Vollständigkeit, Erwartungen des Kunden.

Probleme im RE: Änderungen, Fachsprache, verschiedene Stakeholder, neue Stakeholder, politische/organisatorische Faktoren, unklare Zielvorstellungen.

Anforderungskategorien

Abstraktionsebenen

- **Benutzeranforderungen:** Funktional und nicht-funktional aus Benutzersicht (ohne technisches Detailwissen). Externes Verhalten. Beschreibung aus **Kundensicht**, Teil des **Lastenhefts**. Natürliche Sprache, keine technische Jargon. „**should**“ vs. „**shall**“.
- **Systemanforderungen:** Dienste und Beschränkungen **präzise**. Teil des **Pflichtenhefts**. Für technisches Personal. Sagt aus, **WAS** ein System tun soll (nicht wie). Notationen: strukturierte natürliche Sprache, Pseudocode, Entwurfsbeschreibungssprachen (PDL), grafische Notationen (UML), mathematische Spezifikationen (Z, B, VDM).

Inhaltsebenen

- **Funktionale Anforderungen (FA) – „Was?“**
 - Was soll das System leisten? Welche Dienste anbieten?
 - Beispiel FA1: „Die Bibliotheks-Suchmaschine soll eine Liste aller Bücher zu einem Thema liefern.“
- **Nicht-Funktionale Anforderungen (NFA) – „Wie (gut)?“**
 - Qualitätsanforderungen: Usability, Reliability, Robustness, Performance, Portability, Security, Supportability, Maintainability.
 - Wenn möglich **messbar**.
 - **Oft kritischer als FA.**
 - Betreffen oft **gesamte Architektur**.
 - Beispiel NFA1: „Das Suchergebnis soll nach spätestens 5 Sekunden angezeigt werden.“

Metriken zum Messen von NFAs (Sommerville)

Property	Measure
Speed	Processed transactions/second, User/event response time, Screen refresh time
Size	MByte
Ease of use	Training time, Number of help frames
Reliability	Mean time to failure, Probability of unavailability

Property	Measure
Robustness	Time to restart after failure, % of events causing failure, Probability of data corruption
Portability	% of target-dependent statements, Number of target systems

NFA-Taxonomie (Sommerville – grobe Kategorien)

- **Produkt-Anforderungen:** Usability, Performance/Effizienz, Speicher, Reliability, Robustheit, Portabilität
- **Organisatorische Anforderungen:** Anforderungen an Liefer-Prozess, Implementierung, Standards (z. B. monatliche Reviews der Doku)
- **Externe Anforderungen:** Interoperabilität, ethisch, rechtlich-vertraglich (z. B. Drittel-Zahlung pro Meilenstein)

Qualitätskriterien (IEEE ISO/IEC/IEEE 29148-2011)

1. **Vollständig:** Jede Anforderung beschreibt die zu liefernde Funktionalität vollständig. Messbar. Unvollständige Anforderungen markieren (z. B. "tbd").
2. **Atomar:** Jeder Anforderungssatz enthält **genau eine Anforderung** (Vollverb). Keine Konjunktionen, die mehrere Anforderungen verbinden.
3. **Technisch lösungsneutral:** Beschreibt **was** gefordert wird, nicht **wie**. Keine unnötigen Architekturzwänge, keine Lösungsmöglichkeiten vorab ausschließen.
4. **Notwendig:** Nur Leistungen, die der Kunde tatsächlich benötigt. Zur Erfüllung eines Systemziels. Gültig und aktuell.
5. **Verfolgbar (traceable):** Zu Quellen und nachgelagerten Artefakten verfolgbar. Anforderungsnummerierung.
6. **Realisierbar:** Möglich innerhalb bekannter Fähigkeiten/Grenzen, ohne grundlegende technische Neuerungen, mit annehmbarem Risiko.
7. **Konsistent:** Widerspruchsfrei zu allen anderen Anforderungen.
8. **Eindeutig:** Nur auf eine Weise verstehbar. Einfach formuliert. Subjektive Angaben durch harte Fakten ersetzen.
9. **Prüfbar (testbar):** Funktionalität muss durch Test/Messung nachweisbar sein.

Typische Fehler

- Auslassen einer Beschreibung
- Widerspruch
- Mehrdeutigkeit
- Fehlende Messbarkeit
- „Noise“ (irrelevante Info)
- Überspezifikation
- Machbarkeitsprobleme
- Schlechte Struktur, Unverständlichkeit, fehlende Referenzen, Undurchsichtigkeit

Beispiele: - „Die Zugtüren müssen zwischen Stationen immer geschlossen bleiben. Sie müssen offen sein wenn es einen Nothalt gab.“ → **WIDERSPRUCH**. - „Die Zugtüren müssen geöffnet werden, wenn der Zug in der Station ist.“ → **MEHRDEUTIGKEIT/UNGENAUIGKEIT**. - „Die Informationsanzeige im Zug muss benutzerfreundlich sein.“ → **UNMESSBAR**. - „Jedes Abteil ist mit einem Informations-Panel und einem Nichtraucher-Aufkleber versehen.“ → **NOISE**. - „Die angezeigte Geschwindigkeit muss mindestens 7 km/h über der physikalischen Geschwindigkeit liegen.“ → **UNDURCHSICHTIGKEIT**.

Anforderungserhebung

Wissen über Organisation, Domäne, zukünftiges System, Stakeholder ermitteln.

Kommunikation – Vier-Seiten-Modell (F. Schulz von Thun) Jede Nachricht hat 4 Seiten, die von Sender und Empfänger unterschiedlich interpretiert werden: - **Sachinhalt** (worüber informiere ich) – z. B. „Mit der Suchmaske wird nichts gefunden“ - **Selbstoffenbarung** (was gebe ich von mir kund) – z. B. „Ich bin überfordert“ - **Beziehung** (was halte ich von dir) – z. B. „Du solltest mir helfen“ - **Appell** (was möchte ich erreichen) – z. B. „Bitte überarbeite die Suche“

Beispiel: „Ich finde mit der Bibliothekssuche nichts.“ Diese Aussage kann unterschiedlich gedeutet werden, je nachdem welche Seite betont wird. RE muss die richtige Seite erkennen.

Wissensebenen (nach Kano): - Basis-Faktoren = **unterbewusstes** Wissen → **Beobachtungstechniken, artefaktbasiert** - Begeisterungs-Faktoren = **unbewusstes** Wissen → **Kreativitätstechniken** - Leistungs-Faktoren = **bewusstes** Wissen → **Befragungstechniken**

Wichtig: Immer eine **Kombination** an Techniken verwenden!

Welche Methode macht Sinn? (Chris Rupp) Techniken liegen auf einem Spektrum von **wenig Interaktion** mit Stakeholdern (RE muss Wissen selbst beschaffen) bis **gezielte Interaktion** mit ausgewählten Stakeholdern:

- **Wenig Interaktion:** Hintergrundstudie, Systemarchäologie, Reuse, Audio-/Videoaufzeichnung
- **Mittlere Interaktion:** Feldbeobachtung, Apprenticing, Mind Mapping, Szenarien, Fragebogen
- **Hohe Interaktion:** Brainstorming, 6-3-5, Wechsel der Perspektive, Interview, Workshop

Kreativitätstechniken (unbewusstes Wissen)

- **Brainstorming:** 5–10 Teilnehmer, 20 Minuten, keine Kritik, Moderator. (+) viele Ideen schnell, Weiterspinnen, freie Ideen, innovativ. (–) schwierige Gruppendynamik, Terminkoordination.
 - **Methode 6-3-5:** 6 Teilnehmer, je 3 Ideen, 5 mal weitergeben. (+) gleichmäßige Einbindung, schriftliche Form, räumliche Verteilung. (–) weniger Aktivität untereinander.
 - **Wechsel der Perspektive (6-Hut-Denken):** 6 verschiedene Denkweisen:
 - **Weiß:** objektiv (Zahlen/Fakten)
 - **Rot:** subjektiv (Gefühle, Ängste, Hoffnungen)
 - **Schwarz:** objektiv negativ (Zweifel, Bedenken, Risiken)
 - **Gelb:** objektiv positiv (Chancen, Pluspunkte, Ziele)
 - **Grün:** neue Ideen (beliebige, Brainstorming-ähnlich)
 - **Blau:** Prozesskontrolle (Moderation, Anleitung)
- (+) festgefahrene Sichtweisen aufbrechen. (–) kommt nicht gut bei Introvertierten/Konservativen.

Beobachtungstechniken (unterbewusstes Wissen, Basisfaktoren)

- **Feldbeobachtung:** Verhaltensbeobachtung in „natürlicher Umgebung“. (+) bei automatisierter unbewusster Arbeit, bei schlechter zeitlicher Verfügbarkeit, bei Prozessabweichungen. (–) selten auftretende Abläufe nicht beobachtbar, evtl. anderes Verhalten bei Beobachtung.
- **Apprenticing:** „In die Lehre gehen“, Erleben des Ist-Systems. (+) schwer ausdrückbares Wissen, „Meister“ statt Kontrolle, angstfreier Umgang. (–) nicht in kritischen Umfeldern (Flugsicherung), zeitintensiv, Probleme bei unklarer Stakeholder-Abgrenzung.

Befragungstechniken (bewusstes Wissen, Leistungsfaktoren)

- **Fragebogen:** Vorstudien!, redundante Fragen, offene und geschlossene. (+) schnelle Antworten, niedrige Kosten, große Benutzergruppen. (–) Bias, kein implizites Wissen, schwer für NFA.
- **Interview** (wichtigste Technik!):
 - Vorgehen: Stakeholder auswählen → Treffen organisieren → Fragen + Protokoll → Bericht → Zusenden + Validierung.
 - Strukturiert und unstrukturiert (Kombination).
 - Tipps: Vorbereitung, angenehme Atmosphäre, Aufzeichnungserlaubnis, einfache Fragen starten, Nachfragen.
 - (+) brauchbare Liste, neue Punkte durchs Gespräch, individuell. (–) zeitaufwändig, subjektiv/objektiv trennen, Geschick des Leiters.

Artefakt-basierende Techniken (unterbewusstes Wissen)

- **Hintergrundstudie:** Bestehende Dokumente sammeln/auswerten. (+) Wissen über Organisation/Domäne, Einblick vor Ort. (–) Dokumentenmenge, Meta-Wissen erforderlich, Schriftliches $\neq$ Wirklichkeit.
- **Systemarchäologie:** Auf Basis existierender Systeme/Doku, Benutzerhandbuch, Testfälle, ggf. Code-Recherche. (+) keine Funktionalität verloren. (–) aufwändig, schlecht bei vielen Änderungen, Doku-Qualität.
- **Reuse:** Wiederverwendbare Anforderungen aus anderen Projekten. (+) Kosteneinsparung, evtl. Testfälle/Modell. (–) schwer richtige zu finden, alte Qualität oft schlecht, Fehler-Übernahme.

Unterstützende Techniken

- **Workshop:** (+) Verständnis, Kompromissbereitschaft. (–) Gruppendynamik, räumliche Verteilung.
- **Mind Mapping:** (+) Struktur und Visualisierung. (–) oft nur von Anwesenden interpretierbar.
- **Audio-/Videoaufzeichnung:** (+) kein Infoverlust. (–) zeitaufwändige Nachbereitung, Beobachtungsgefühl.
- **Szenarien:** (+) leicht verständlich, zur Validierung wiederverwendbar. (–) keine NFA-Doku, unvollständig.

Anforderungsdokumentation

WO werden Anforderungen gesammelt?

- **Lastenheft:**
 - Ziel: Einholen von Angeboten. „Vom Auftraggeber festgelegte Gesamtheit der Forderungen an die Lieferungen und Leistungen eines Auftragnehmers“ (DIN 69901-5).
 - **WAS und WARUM**, keine Umsetzungsvorschläge, **natürliche Sprache**.
 - Inhalte: Einführung, Ist-Zustand, Soll-Zustand, Schnittstellen, FA und NFA (Qualitätskriterien), Risiken/Akzeptanz, Abnahmekriterien.
- **Pflichtenheft:**
 - „Vom Auftragnehmer erarbeitete Realisierungsvorgaben aufgrund der Umsetzung des vom Auftraggeber vorgegebenen Lastenhefts“ (DIN 69901-5).
 - **Konkrete Realisierung** bzw. Implementierungsbeschränkungen, **technische Sprache**.
 - Muss leicht veränderbar sein, **Referenz für Wartungspersonal**.
 - Empfohlene Inhalte (IEEE/ANSI 830-1993): Vorwort, Einleitung, Begriffe, Benutzeranforderungen, Systemarchitektur, Spezifikation Systemanforderungen, Systemmodelle, Systementwicklung, Anhänge, Index.
- **Product Backlog:** Liste aller Projektarbeiten, priorisiert, oben = genauer spezifiziert. **Jedes Backlog Item ist eine User Story.**

WIE werden Anforderungen dokumentiert?

- **Prosa:** natürliche Sprache, häufigste Technik. (+) keine komplizierte Notation. (–) Mehrdeutigkeit, Unvollständigkeit.
- **User Stories:** „Als möchte ich , so dass ich .“ Vorderseite: Priorisierung, Name, Beschreibung (WER, WAS, WARUM), Risiko, Story Points. Rückseite: Akzeptanzkriterien („Angenommen , wenn , dann .“).
- **Use Case Diagramm:** Übersicht über Funktionalität, mehrere Use-Cases mit Beziehungen. (+) leicht verständlich, akzeptiert für Skizzen. (–) keine NFAs, Redundanzen/Inkonsistenzen.
- **Use Case Beschreibung:** Genaue Beschreibung in natürlicher Sprache, NFA direkt in Use Case oder separat (Zusatzanforderungen). Template/Formular. (+) Formular hilft, Struktur, Erfolgs-/Misserfolgsszenarien. (–) Aufblähung mit Alternativabläufen.
 - Template-Felder: ID, Name, Beschreibung, beteiligte Akteure, Status, verwendete Use Cases (includes), Auslöser, Vor-/Nachbedingungen, Nachbedingungen Fehlerfall, Invarianten, Hauptszenario, Alternativszenarien.
- **Aktivitätsdiagramm:** (+) Detailgrad variabel, Alternativen/Entscheidungen, Erfolgs-/Misserfolg, leicht erlernbar. (–) Komplexität bei vielen Notationen.
- **Sequenzdiagramm:** (+) intuitive zeitliche Abfolgen, Kommunikationsprotokoll. (–) tendiert zur Unübersichtlichkeit.

Anforderungvalidierung

Reviews

Typ	Beschreibung
Stellungnahme	Kollege bekommt Anforderungen vom Autor, markiert Auffälligkeiten. (+) 4 Augen, Lücken, geringer Aufwand. (–) klare Prüfziele nötig, kein Klären von Verständnis.
Walkthrough	Moderator ($\neq$ Stakeholder), Prüfer, Autor. Autor präsentiert + Gedankengänge. Regeln festlegen, ggf. mit Prototyp. (+) gemeinsames Verständnis. (–) Ablenkung möglich, abhängig vom Leiter.

Typ	Beschreibung
Inspektion	Auch auf Testfälle/Code anwendbar. 6 Phasen: 1. Planung (Checklisten, Inspektorengruppe), 2. Vorbesprechung, 3. Individuelle Vorbereitung, 4. Reviewsitzung, 5. Nachbereitung/Bewertung, 6. Ende. (+) viele Auffälligkeiten, formal, Checklisten. (–) zeitaufwändig, kostenintensiv, abhängig von Erfahrung.

Prototyping / Simulationsmodell

- Throw-away vs. **evolutionäre Prototypen**.
- Trainingsobjekt bzw. Validierung, Usability.
- (+) Vorstellungskraft, Diskussion, frühe Implikationen, in jeder Phase. (–) aufwendig, Spielereien, Aussehen $\neq$ Endsystem, “fast fertig”-Gedanke, nicht alle Aspekte.

Testfälle

- Erstellung im Rahmen vom RE.
- **Natürlichsprachlich:** Ausgangssituation – Testereignis – erwartetes Ergebnis.
- **Anforderungsautor $\neq$ Testfallersteller**, Prüfung der Testfälle durch Autor.
- (+) Sichtwechsel, Basis für Automatisierung, Auffälligkeiten. (–) zeitaufwändig, Vorwissen nötig, nicht alle Qualitätskriterien prüfbar.

Hilfsmittel: Lesetechniken

Lesetechnik	Fokus	Verwendung
Ad-hoc-Lesen	einzelne Anforderungen, bestimmte Aspekte	Stellungnahme
Ablauforientiertes Lesen	Verhalten in einem Szenario	Walkthrough
Schrittweise Abstraktion	Gemeinsames Ziel von Anforderungen	z. B. Notwendigkeit prüfen
Perspektivbasiertes Lesen	aus Sichtweise (User, Tester, Entwickler)	vollständiges Bild

Checklisten

- Vorgegebene Kriterien/Fragen (Standardisierte Struktur? Eindeutige IDs? Quelle? Technische Infos? Verschieden interpretierbar?).
- Checkliste mit Team erstellen, **je nach Fokus** verschiedene Checklisten.

Anforderungsabstimmung (bei Konflikten)

4 Schritte: **Identifikation** → **Analyse** → **Auflösen** → **Dokumentation**.

- Identifikation: Indikatoren in Kommunikation und Dokumentation.
- Auflösen: **Abstimmungs-/Weisungsmethoden** oder **Annäherungsmethoden** (Durchsetzung vs. Kooperation).
- Dokumentation: Worum? Wurde aufgelöst? Warum und wie? → Nachvollziehbarkeit.

Requirements Management

“Umfasst alle Maßnahmen, die notwendig sind, um Anforderungen und anforderungsbezogene Artefakte zu dokumentieren, zu ändern und nachzuverfolgen.” [CPRE2012]

Fragen: Wer gibt wann wem was? Wer darf wann was? Was hängt wie mit was zusammen? Wie läuft's?

Support: - **Requirements-Engineering Leitfadens:** zentrales Artefakt mit allen Prozessen, Methoden, Artefakten, Tools, Rollen. - **Gliederungsstrukturen:** Standards (Volere, IEEE 29148-2011, V-Modell XT), Gliederungsstrategien (lexikalisch nach Definitionen, Use Cases/Features, technischen Anforderungen, Verantwortungs-

bereich/Prozessschritten), eindeutige **Objekt-IDs** (US-Bib-008, TF-Bib-008), **Duplikate vermeiden**. - **Tools**: Jira, IBM Doors, Redmine, etc.

6. QUALITÄTSSICHERUNG

Definitionen

Qualität (ISO 9000): Gesamtheit von Merkmalen einer Einheit bezüglich ihrer Eignung, festgelegte und vorausgesetzte Erfordernisse zu erfüllen.

3 Sichten: - Zufriedenheit der Auftraggeber/Anwender. - Erfüllung von Anforderungen, Attributen, Systemfunktionen. - Erfüllung von Richtlinien, Vorgaben, Standards, Regelungen.

Qualitätssicherung (QS) [IEEE 610.12]: 1. Geplantes und systematisches Muster aller Aktionen, um angemessenes Vertrauen zu geben, dass ein Produkt etablierten technischen Anforderungen entspricht. 2. Aktivitäten zur Bewertung des Prozesses, durch den Produkte entwickelt/hergestellt werden.

Qualitätsmanagement: Planung, Lenkung, Sicherung, Anpassung (PDCA).

Qualitätsfaktoren (ISO/IEC 9126:1990 → ISO/IEC 25010:2011)

- **Functionality**: Angemessenheit, Richtigkeit, Ordnungsmäßigkeit. „Vorhandensein von Attributen mit festgelegten Eigenschaften“.
- **Efficiency**: Zeitverhalten, Verbrauchsverhalten, Konformität. „Verhältnis zwischen Leistung und eingesetzten Betriebsmitteln“.
- **Compatibility** (neu in 25010): Koexistenz, Interoperabilität, Konformität. „Funktionalität bzw. Informationsaustausch mit anderen Produkten“.
- **Usability**: Verständlichkeit, Erlernbarkeit, Bedienbarkeit, Attraktivität, Konformität. „Aufwand für das Benutzen“.
- **Reliability**: Reife, Fehlertoleranz, Robustheit, Wiederherstellbarkeit, Konformität. „Kann das System seinen Dienst über bestimmten Zeitraum anbieten?“
- **Security** (neu in 25010): Vertraulichkeit, Integrität, Nicht-Abstreitbarkeit, Verantwortlichkeit, Authentizität, Konformität.
- **Maintainability**: Analysierbarkeit, Modifizierbarkeit, Stabilität, Testbarkeit. „Aufwand zur Durchführung von Änderungen“.
- **Portability**: Anpassbarkeit, Installierbarkeit, Koexistenz, Austauschbarkeit, Konformität. „Aufwand, um SW in neues System zu integrieren“.

Fehler im Code

- Space Shuttle Software: < 1 Bug pro 10.000 LOC.
- Microsoft interne Regel: ~ einige Bugs / 1.000 LOC.
- Standard-Software: bis ~ 30 Bugs / 1.000 LOC.

Fehler finden ist teuer – Optimum zwischen Fehlerkosten, Fehlervermeidungskosten und Qualität.

Methoden der QS

- **Analytische QS**: Bewertung der Qualität eines Produkts.
- **Konstruktive QS**: Mängel von vornherein vermeiden, Niveau erreichen/steigern.
- **Organisatorische QS**: Rahmenbedingungen, Lenkung der Qualität.

Analytische QS

Statische QS (Ziel: innere Qualität – Struktur, Design)

- Einsatz in **frühen Phasen** möglich.
- Prüfung **ohne dynamische Ausführung**.
- Repräsentationen (Dokumentation, Code).
- **Methoden**: Metriken und Reviews.

Metriken (IEEE 1061) “Eine SW-Qualitätsmetrik ist eine Funktion, die eine Software Einheit in einen Zahlenwert abbildet.”

- Quantifizieren Qualität, Prognosen (Kosten, Termine, Ressourcen), Entscheidungsunterstützung.
- Unterscheidung: **Kosten-, Fehler-, Umfangs-, Qualitätsmetriken**.
- **GQM-Ansatz** zur Findung geeigneter Metriken: 3 Schritte.

1. **Goals** definieren
2. **Questions** ableiten (bezüglich Ziele)
3. **Metrics** festlegen für jede Frage

Reviews (statisch)

- Stellungnahme, Walkthrough, **Inspektion** (siehe Anforderungsvalidierung – auf Code/Testfälle anwendbar).

Dynamische QS (Ziel: Fehlerfindung in existierenden Teillösungen)

- Überprüfung von Komponenten/Systemen durch **Software Testen**.
- **Verifikation vs. Validation**:
 - **Verifikation**: “Bauen wir das Produkt richtig?” (gegen Spezifikation).
 - **Validation**: “Bauen wir das richtige Produkt?” (gegen Kundenwünsche).

Software Fehler-Begriffe

- **Software Error (Fehler)**: Eine menschliche Aktion, die ein falsches Ergebnis erzeugt.
- **Software Fault (Fehlerfall)**: Ein inkorrektter Schritt, Prozess oder eine Datendefinition im Programm.
- **Software Failure (Fehlverhalten)**: Die Unfähigkeit eines Systems/einer Komponente, die geforderten Funktionen innerhalb spezifizierter Performance-Anforderungen auszuführen.

Software Testen Dijkstra: “Program testing can be a very effective way to show the presence of bugs, but it is hopelessly inadequate for showing their absence.”

- **Black-Box Testen**: Tests ohne Code-Kenntnis.
- **White-Box Testen**: Tests mit Code-Kenntnis.
- **Äquivalenzklassenzerlegung**.
- **Grenzwertanalyse**.
- **Mocking**.
- **Testüberdeckungen**:
 - Statement (Anweisungsüberdeckung)
 - Branch (Zweigüberdeckung)
 - Einfache Bedingungsüberdeckung
 - Mehrfache Bedingungsüberdeckung
 - Path (Pfadüberdeckung)

Hilfsmittel

- **Testautomatisierung** (CI, Regressionstests).
- **Code Coverage Analyse**.
- **Code Quality Checks**.

Konstruktive QS (Ziel: Mängel vermeiden, Qualitätsniveau erreichen)

- **Technische Maßnahmen**: Methoden, Sprachen, Werkzeuge.
- **Organisatorische Maßnahmen**: SE-Prozessmodelle.
- **Menschliche Maßnahmen**: Schulungen.

Organisatorische QS (Ziel: Rahmenbedingungen, Lenkung)

- **Wissensmanagement**
- **Templates/Checklisten**
- **Standards** (Software, Produkt, Prozess) – Teil des Qualitätsmanagements.

Standards

- **ISO** = International Standardization Organization
- **IEEE** = Institute for Electrical and Electronic Engineering
- **SEI** = Software Engineering Institute (Carnegie Mellon University)

- **ISO 9000**-Familie für Qualitätsmanagementsysteme:
 - **ISO 9001**: Organisationen, die Produkte entwerfen/entwickeln/warten (aktuell **ISO 9001:2015**).
 - **ISO 9001-3**: Normen für SW-Entwicklung (QA-Aktivitäten, Mgmt, Tools, Messungen).
- **ISO/IEC 250xx**: Qualitätskriterien und Bewertung von SW-Produkten.
- **IEEE Standard Familie**: ~ 160 Dokumente zum Thema SW-Engineering.
- **SEI CMMI-DEV** (Capability Maturity Model Integration for Development): Prozessverbesserungsansatz, Sammlung von „best practices“.

CMMI – 5 Reifegrad-Stufen

1. **Initial** (initialer Prozess): ad-hoc, SW-Entwicklung “irgendwie”, Erfolg nicht erklärbar.
2. **Repeatable** (wiederholbarer Prozess): Planung, Meilensteine, QS, Erfolg hängt von Personen ab.
3. **Defined** (definierter Prozess): Konstruktive QS, **institutionalisierter Prozess**, Qualität meist gut.
4. **Managed** (gesteuerter Prozess): durch Messen gesteuert, vorhersagbar hohe Qualität.
5. **Optimizing** (selbstverbessernder Prozess): kontinuierliches Feedback zur Verbesserung.

7. PROJEKTMANAGEMENT

Definition (DIN 69901)

“Projektmanagement ist die Gesamtheit von Führungsaufgaben, -organisation, -techniken und -mitteln für die Abwicklung eines Projekts.”

Projekt (H. Karnovsky): einmaliges Vorhaben mit definiertem Anfang, definiertem Ende und mehreren beteiligten Personen ($\neq$ Tagesgeschäft, $\neq$ Produktion).

Das „magische“ Dreieck

Vier Dimensionen, die zusammen verwaltet werden müssen: **Zeit – Kosten – Qualität – Funktionalität**.

Projektgröße (Karnovsky)

- **MINI**: 1–2 Personen (z. B. Kundenverwaltung).
- **MIDI**: 3–20 Personen (z. B. Webapplikation).
- **MAXI**: > 20 Personen (z. B. verteilte Anwendung).
- Risiken: Domänenkenntnis, Komplexität, Qualifikation MA, Technologien.

Rollen im Projekt

- **Auftraggeber** $\leftrightarrow$ **Projektleiter/Teamkoordinator** $\leftrightarrow$ **Entwicklerteam**
 - Projektleiter „redet mit“ Auftraggeber, „koordiniert“ Team.
 - Team: SW-Architekt, Doku-Beauftragter, Test-Beauftragter, Stellvertreter, ...

Organisationsstrukturen

Stab-Projektorganisation

- Abteilungen mit Spezialisierungen.
- (+) effiziente Nutzung techn. Personal, Karrieremöglichkeiten, Technologietransfer, Stabilität.
- (–) schlechte Kunden-Schnittstelle, weniger Projektautorität, schlechte horizontale Kommunikation, eher diszipliniert, langsame Workflows.

Reine Projektorganisation

- MA werden „abgezogen“ für Projekte.
- (+) Planbarkeit, Kostenkontrolle, Ansprechpartner Kunde, schnelle Reaktion.
- (–) ineffizient bei Experten, unsicheres Weiterkommen, wenig Wissensweitergabe.

Matrix-Projektorganisation

- Balance zwischen reiner und Stab-Organisation.
- (+) effizienterer Ressourcen-Einsatz.
- (–) MA haben mehr als einen Vorgesetzten, Zeitmanagement.

Projektphasen (unabhängig von Organisationsform und Typ)

1. **Projektdefinition:** Projektvorschlag, -entscheidung, -auftrag.
2. **Projektplanung:** Projektstruktur, Aufwände schätzen, technische/wirtschaftliche Planung.
3. **Projektverfolgung:** Projektüberwachung, Aufwandserfassung, Meilenstein-Trend-Analyse.
4. **Projektabschluss:** Übergabe der Ergebnisse, Entlastung des Teams, Abrechnung der Aufwände, „lessons learned“-Bericht.

Aufwandsschätzung

Wozu?

- Kalkulation, Angebotslegung.
- Selber bauen oder kaufen?
- Personalplanung.
- Nachkalkulation.

Schätztrichter (Böhm 2000) Schätzungen werden umso **genauer, je näher** man dem Projektende ist. Anfangs hoher Schätzfehler, der mit der Zeit sinkt.

Schätzverfahren – 3 Kategorien

A) Empirische Schätzverfahren Erfahrungsbasiert, gut bei ähnlichen Projekten. (+) einfach, billig. (–) grobe Fehler möglich.

Techniken: - **Expertenschätzung:** Mehrere Experten in Meetings. (+) einfach, günstig. (–) grobe Fehler möglich.
- **Delphi-Methode:** Mehrere Runden, **konvergiert**. 1. Übergabe Projektbeschreibung und Unterlagen. 2. Treffen + Diskussion. 3. **Anonymes** Schätzen. 4. Sammlung + Anonymisierung durch Koordinator. 5. Diskussion der Ergebnisse. 6. Wiederholung bei Schritt 3, bis Koordinator zufrieden.

(+) sehr gute Ergebnisse, vermeidet Einfluss dominanter Teilnehmer, eliminiert Ausreißer. (–) Erfahrung nötig, langwierig. - **Analogieschätzung:** Auf Basis ähnlicher Projekte. Idealerweise selbes Geschäftsumfeld, ähnliches System, gleiche Methoden/Sprachen. Adjustierungsfaktoren. (+) zuverlässig bei Erfahrung, schnell/früh. (–) Unterschiede evtl. größer als gedacht. - **Bottom-Up-Methode:** Jede Komponente einzeln. **Immer mit anderer Methode kombinieren.** (+) Ableitung von Meilensteinen. (–) Aufwand nicht einfach addierbar.

B) Algorithmische Schätzverfahren Berechnung von Kosten-/Durchlaufzeit-Funktionen. Modell muss kalibriert sein. (+) beste Prognose bei richtiger Kalibrierung. (–) ohne Maßzahlen vergangener Projekte unzuverlässig.

Function Points (IBM, A. Albrecht 1979)

Basis: Anzahl der zu implementierenden Funktionen, aus **Benutzersicht**.

Vorgehensweise: 1. **Sammeln und Kategorisieren** der Daten in 5 Kategorien: - **EI** (External Input) – Eingaben - **EO** (External Output) – Ausgaben (zuvor bearbeitete Daten) - **EQ** (External Inquiry) – Abfragen (Auflistung, Suchergebnis) - **ILF** (Internal Logical File) – interne Datenbestände - **EIF** (External Interface File) – externe Datenbestände (Lesezugriff) 2. **Klassifizierung** (niedrig, mittel, hoch). 3. **UFP** (Unadjusted Function Points) berechnen: - EI: $\times 3/4/6$ - EO: $\times 4/5/7$ - EQ: $\times 3/4/6$ - ILF: $\times 7/10/15$ - EIF: $\times 5/7/10$ 4. **14 Einflussfaktoren** gewichten (0 = kein, 5 = stark): Datenkommunikation, Verteilung, Geschwindigkeit, HW-Auslastung, Transaktionsrate, Echtzeit-Eingabe, Bedienerfreundlichkeit, Echtzeit-Pflege, Prozesslogik, Wiederverwendbarkeit, Installation, Automatisierung, Vernetzung, Anpassbarkeit. $\rightarrow$ **TDI** (Total Degree of Influence) = $\sum SMI$. 5. **Korrekturfaktor VAF** = $0.65 + 0.01 \times TDI$. 6. **FP** = $UFP \times VAF$.

Weitere Projektgrößen (grob, Kalibrierung nötig): - **TDEV** (Time for Development in Months) = $FP^{0.4}$ - **DEV** (Number of Developers) = $FP / 150$ - **E** (Effort) = $TDEV \times DEV$

Application Points

Schätzung über Objekte: - Anzahl unterschiedliche **Screens** $\times 1/2/3$ (niedrig/mittel/hoch) - Anzahl **Reports** $\times 2/5/8$ - Anzahl **Module** $\times 10$ - = Application Points (AP) - **NAP** = $AP \times (100 - RC) \times 0.01$ (RC = Reuse). - Merkmale **M1** (Erfahrung MA) und **M2** (Reife CASE Umgebung) $\rightarrow$ Mittelwert $\rightarrow$ **PROD**: - 1 $\rightarrow 4$ - 2 $\rightarrow 7$ - 3 $\rightarrow 13$ - 4 $\rightarrow 25$ - 5 $\rightarrow 50$ - **PM** (Personenmonate) = $NAP / PROD$.

COCOMO 81 (Boehm 1981) und COCOMO II (2000)

Basis: **KDSI** (Kilo Delivered Source Instructions). E in Personenmonaten (1 PM = 19 Arbeitstage), TDEV in Kalendermonaten (CM). Wasserfall $\rightarrow$ Korrekturfaktoren 0,4 und 0,7.

3 Präzisionsstufen: - **Basic:** nur Anzahl KDSI, $S = \sum KDSI$. - **Intermediate:** Schätzung auf Subsystem-Ebene, Korrekturfaktor $M = \prod M_i$. - **Detailed:** weitere Aufspaltung der Korrekturfaktoren.

3 Projektarten: - **Organic** (< 50 KDSI, kleine Programmgröße): - $E(\text{Basic}) = 2.4 \times S^{1.05}$ - $E(\text{Intermediate}) = M \times 3.2 \times S^{1.05}$ - $TDEV = 2.5 \times E[PM]^{0.38}$ - **Semidetached** (< 300 KDSI): - $E(\text{Basic}) = 3.0 \times S^{1.12}$ - $E(\text{Intermediate}) = M \times 3.0 \times S^{1.12}$ - $TDEV = 2.5 \times E[PM]^{0.35}$ - **Embedded** (für neuen Einsatzbereich): - $E(\text{Basic}) = 3.6 \times S^{1.20}$ - $E(\text{Intermediate}) = M \times 2.8 \times S^{1.20}$ - $TDEV = 2.5 \times E[PM]^{0.32}$

Intermediate Korrekturfaktoren (M_i): RELY, CPLX, TIME, ACAP (Analyst Capability), PCAP (Programmer Capability), LEXP (Language Experience), TOOL, SCED (Schedule). Werte je nach Bewertung (sehr niedrig bis extrem hoch).

Beispiel: 3 Subsysteme (Kernsystem 2.1, Datenimport 0.9, Report-Generator 0.6 KDSI), Organic, sehr hohe Verarbeitungsgeschwindigkeit (TIME=1.30), wenig Tools (TOOL=1.24), neue Sprache (LEXP=1.07). - Basic: $E = 2.4 \times 3.6^{1.05} = 9.21 \text{ PM}$; $TDEV = 2.5 \times 9.21^{0.38} = 5.81 \text{ CM}$. - Intermediate: $E = (1.30 \times 1.24 \times 1.07) \times 3.2 \times 3.6^{1.05} = 21.18 \text{ PM}$; $TDEV = 2.5 \times 21.18^{0.38} = 7.98 \text{ CM}$.

COCOMO II (2000): Erweiterung um neue DBMS, CASE Tools etc. **KSLOC** (Kilo Source Lines of Code). 5 Skalierungsfaktoren (Präzedenz, Flexibilität, Risiko-Umgang, Zusammenarbeit, Reife des Prozesses), 17 Kostenfaktoren.

C) Sonstige Schätzverfahren

- **Koste-es-was-es-wolle:** So tief schätzen, dass man Zuschlag bekommt.
- **Schmerzschwelle:** Abtasten, wie viel Auftraggeber zahlen will.
- **Parkinsonsches Gesetz:** Der Aufwand passt sich der vorhandenen Kapazität an.

Planung und Steuerung

Projektstruktur

Work-Breakdown Structure (WBS) / Projektstrukturplan (PSP)

- **Hierarchische Struktur / Baum.**
- 2 Typen: **objektorientiert** und **ablauforientiert**.
- Testen ist auch „Teilprojekt“.
- Grundlage für **Kostenschätzung**.
- **Kleinste plan- und verfolgbare Einheit** = Arbeitspaket.
- Horizontale vs. vertikale Rolle.

Project-Breakdown Structure (PBS)

- Nach **PRINCE2** (PRojects IN Controlled Environments).
- **Zerteilung des Projekts in Teilprodukte.**
- Ergänzendes **Product Flow Diagram** mit Abhängigkeiten.

Planungstechniken

Netzplan / Balkendiagramm

- Graphisches Planungsinstrument.
- Abhängigkeiten und Bearbeitungsreihenfolge.
- Grundlage: WBS.
- **Einsatz** bei Projekten nach Wasserfall/V-Modell.

PERT-Diagramm (Program Evaluation and Review Technique) Knoten enthält: Aktivität, Dauer, **Frühster Anfang (FA)**, **Frühstes Ende (FE)** = FA + Dauer, Spätester Anfang/Ende. - FA = Maximum FE der Vorgänger. - Abhängigkeiten als Pfeile.

GANTT-Diagramm (Henry Gantt)

- Konkrete Aktivität / Arbeitspaket.
- Zeitlicher Aufwand = Dauer.

- FA – FE, Abhängigkeiten, **Puffer**.
- Aktive Arbeitspakete zu bestimmtem Zeitpunkt.

Burndown Chart (siehe SCRUM)

Cumulative Flow Diagram (CFD) Visualisiert Fluss der Tasks durch Workflow-Status über die Zeit.

Steuerung / Verfolgung

Meilenstein-Trend-Analyse (MTA)

- Vergleich geplante vs. tatsächliche Meilensteine über Zeit.
- Linien zeigen: OK, Problem, Puffer.

Kosten-Trend-Analyse (CTA)

- Gegenüberstellung von **Soll-Kosten** und **Ist-Kosten**.
- Problemphasen vs. zu viel Puffer.

Personal- und Risikomanagement

Personalmanagement

- **Mitarbeiter:** Wertschätzung, Einkommen, Verantwortung, Weiterbildung. **Motivation:** aufgabenorientiert, selbstorientiert, interaktionsorientiert.
- **Teamarbeit:** Gruppenzusammensetzung (Persönlichkeiten, Erfahrung, Geschlecht), Gruppengröße, Arbeitsumgebung.
- **Projekt:** „Adding manpower to a late software project makes it later“ (Fred Brooks, 1975).
 - Messen in Mann-Monaten ist Mythos.
 - Einarbeitungszeit beachten.
 - Integration ins Team.
 - **Personal früh aufstocken.**

Risikomanagement Risiko [A. Schatten]: Ereignis, das mit abschätzbarer Wahrscheinlichkeit auftreten kann und dem Projekterfolg erheblichen und quantifizierbaren Schaden zufügen kann.

3 Arten: - **Projektrisiken:** Zeitplan, Budget. - **Produktrisiken:** Qualität, Leistung. - **Wirtschaftliche Risiken:** Auswirkungen aufs Unternehmen.

Vorgehen (Sommerville): 1. **Risikoidentifizierung:** Checklisten (Technologie, Personal, Unternehmen, Tools, Anforderungen, Schätzungen), Komponenten-Prüfung, Risiko-Bäume, „Was wäre wenn?“. 2. **Risikoanalyse:** Wahrscheinlichkeit + Schweregrad. Qualitativ (Bewertungstabelle) oder quantitativ (Gefährdungspotential).

Beispiel-Risiko-Bewertungstabelle (Zugtür-Fehler):

Konsequenz	Wahrscheinlich	Möglich	Unwahrscheinlich
Verlust von Leben	katastrophal	katastrophal	schwerwiegend
Verletzungen	katastrophal	schwerwiegend	hoch
Zuggarnitur beschädigt	hoch	moderat	niedrig
Reputationsverlust	moderat	niedrig	niedrig

3. Risikoplanung:

- **Vermeidung** (z. B. defekte Komponenten durch zugekaufte ersetzen).
- **Minimierung** (z. B. mehr Überschneidungen MA wegen Krankheit).
- **Notfallplan** (z. B. bei finanziellen Schwierigkeiten).

4. Risikoüberwachung: Messen, Bewerten, Anpassen.

Laws of Project Management (Shtub, humorvoll)

- Kein Projekt schließt wie ursprünglich geplant ab.
- Projektfortschritt erreicht schnell 90 % und bleibt dann bei 90 %.
- Ungenaue Projektziele ersparen die Peinlichkeit einer genauen Kostenschätzung.
- Wenn alles gut läuft, geht etwas schief.
- Wenn es nicht schlimmer kommen kann, kommt es schlimmer.
- Wenn die Dinge besser scheinen, hat man etwas übersehen.
- Erlaubte Änderungen → Änderungsrate übersteigt Produktivitätsrate.
- Kein System ist fehlerfrei; Fehler beheben erzeugt neue Fehler.
- Achtlos geplantes Projekt: $3 \times$ so lang. Sorgfältig geplant: $2 \times$ so lang.
- Teams hassen Fortschrittsberichte (zeigt langsamen Fortschritt).

8. KONFIGURATIONSMANAGEMENT

Motivation

- Software ändert sich fortlaufend (Bugfixes, neue Anforderungen, Systemumgebung).
- Änderungen → **neue Version**.
- “Welche Version hat den Bug behoben? Welche das Feature eingeführt?”

Definition

Software Configuration Management (SCM): Anwendung (und Entwicklung) von Verfahren (Standards), Prozessen und Tools zur **Verwaltung sich weiterentwickelnder Systemprodukte** [Sommerville]. - Manchmal Teil des Qualitätsmanagements. - Standard: **IEEE 828-1990**. - Unterstützt Entwicklung in Teams (Überschneidungen vermeiden).

Aktivitäten/Themen (3 Bereiche)

1. **Änderungsmanagement:** Erfassen + Abschätzen von Änderungen (Realisierbarkeit, Kosten). Wie reagieren Prozesse auf Änderungen?
2. **Versionsmanagement:** Verwalten verschiedener Versionen, Vermeidung Überschneidungen.
3. **Automatisierung:** Build Management, CI/CD, DevOps.

Änderungsmanagement

Klassische Sicht Algorithmus für kontrollierte Änderungen: 1. Änderungsantrag stellen. 2. Analyse. 3. Wenn Änderung notwendig: Implementierung bewerten, Kosten einschätzen, in DB speichern, an **Änderungskommission**. 4. Wenn akzeptiert: Änderung durchführen, mit Bezug speichern, zum Test, bis Qualität ausreichend. 5. Sonst ablehnen.

Änderungskommission Beurteilt aus **strategischer und organisatorischer Sicht** (nicht primär technisch):
- Konsequenzen ohne Änderung? - Erwarteter Nutzen? - Wie viele Benutzer betroffen? - Kosten? - Releaseplan
- soll noch im nächsten Release?

Agile Sicht

- Änderungsanforderungen wie gewöhnliche Anforderungen behandeln.
- Als **User Story ins Product Backlog**.
- Planung nach Priorität in einem Sprint.
- Umsetzung im Sprint.
- Auslieferung nach Sprintende.
- Sofortige Änderung → wie Produktionssupport (Sprintende, Reparatur).
- Agile Methoden involvieren den **Kunden** in den Entscheidungsprozess.

Versionsmanagement

Motivation

- Große Teams.
- Fehler → lokale Änderungen rückgängig machen.
- Verständnis der Abläufe, Zusammenarbeit.
- Sehr schnelle, komplexe Entwicklung.

Versionsverwaltung – 3 Arten

- **Zentral:** Client/Server, ein Server. Beispiele: **CVS, SVN**.
- **Dezentral:** Repository pro Client, eigener Entwicklungszweig. Beispiel: **Git**.
- **Lokal:** nur für lokale Entwicklung/Versionierung.

Git – Konzepte Ein üblicher Ablauf: - **clone:** Repository vom Server downloaden, kopiert komplettes Repo. - **commit:** Änderungen lokal speichern. Revisionen über **SHA-1 Hash**. Jede Änderung kurz kommentieren (Commit Message). - **push:** lokale Änderungen zum Server. Repo muss "sauber" sein, sonst zuerst **pull**. - **pull:** Änderungen vom Server holen, in lokalen Stand integrieren (**merge**). Manche Änderungen nicht automatisch zusammenführbar → **Konflikt**.

Zusätzliche Aktionen: - **branch:** neue Verzweigung, vermeidet Merge-Konflikte, am Ende in Hauptstrang integrieren. - **tag:** Revision markieren (z. B. Release „4.2.3“).

Fortgeschritten: rebase, stash, checkout, diff, ...

Commit Messages – Rules

1. **Zusammenfassung und Details trennen** (Message Body oft nicht sichtbar).
2. **Zusammenfassung auf 50 Zeichen begrenzen** (GitHub trennt nach 72 Zeichen).
3. **Referenzen angeben** (z. B. Links zum Issue Tracker, [MYISSUE-1234]).

Tangled Commits

- Commit beinhaltet Änderungen mehrerer Tasks → „**Verknäult**“.
- Schlecht: Retrospektive schwer, Fehler suchen schwer.
- **Best Practice:** 1 Commit pro Task/Schritt, sinnvolle Message, Referenz zum Issue ([ISSUE-1234]).
- Forschung: Faktoren für Tangling: zeitliche/örtliche Nähe, gleiche Klasse, gemeinsame Variablenzugriffe. (Etwas) Tool Support.

Merging – Lösungsstrategien

- **Lock-Modify-Unlock:** Datei beim Auschecken sperren. (–) lange Wartezeiten.
- **Copy-Modify-Merge:** jederzeit Auschecken; Konflikt evtl. beim Einchecken. (–) nicht immer automatisch.
- **Feature Branches:** in eigenem Branch arbeiten, am Ende integrieren.

Merge-Konflikt-Marker in Git: <<<<<<<, =====, >>>>>>> – manuell auflösen.

Git-Branch-Modell (für SE2-Übung empfohlen)

- **Master Branch:** stabiler Code.
- **Feature Branches:** für jede neue Funktionalität.
- **Release Branch:** für Releases.
- **Hotfix Branch:** für kritische Fixes.

Semantic Versioning (MAJOR.MINOR.PATCH)

- **MAJOR:** erhöhen bei **inkompatiblen API-Änderungen**.
- **MINOR:** hinzufügen von Funktionalität **abwärtskompatibel**.
- **PATCH:** kleine Änderungen, **abwärtskompatible Bug Fixes**.

Postfixes: - **-SNAPSHOT:** während Entwicklung, häufige Änderungen (1.2.1-SNAPSHOT). - **-M<Zahl>:** Milestone Releases (2.4.1-M1). - **-Beta** oder **-b1:** Beta Testversion. - **-RC** oder **-CR:** Release Candidate. - **.GA:** General Availability (öffentlich). - **Release:** finale Version.

Release Notes Häufige Inhalte: Versionsnummer, Datum, Zusammenfassung, Fixed Bugs, neue Features, Abhängigkeiten, Issue-Referenzen (gh-xxx), Contributors, Download-Links/Dependency-Definitionen.

Buildmanagement / Automatisierung

Motivation

- Viele Branches → viele Versionen, die kompiliert, getestet, deployed werden.
- Ohne Automatisierung nur für kleine Projekte machbar.
- Amazon: 136.000 Builds/Deployments pro Tag.

Software Automation Pipeline

Code → Build → Unit Tests → Integrate → Integr. Tests → Release → Accept. Tests → Deploy → Oper.

Development Integration QA Operation

Feedback loops

- Jede Phase automatisierbar.
- Weiterer Schritt nur wenn Bedingung erfüllt; sonst Feedback Loop.

Abschnitte der Pipeline

- **Build Automation:** individuelle Komponenten bauen, Unit Tests. Typisch vom Entwickler.
- **Continuous Integration (CI):** automatisches Bauen, Testen, Integrieren, Integrationstests. Am CI-Server.
- **Continuous Delivery (CD):** zusätzlich Releases erstellen, auf Test-Umgebungen, Akzeptanztests. **Release ist bereit für Produktivumgebung.**
- **Continuous Deployment (CD):** Release **automatisch** auf Produktivumgebung.
- **DevOps:** Betrieb automatisch (Backups, Monitoring, Scaling).

Continuous Delivery vs. Continuous Deployment

- **Delivery:** finaler Schritt (Deployment) wird **vom Menschen ausgelöst** (zu fixen Zeitpunkten/nach Änderungen).
- **Deployment:** jede erfolgreich getestete Änderung wird **automatisch** in Produktion ausgebracht.

Build Automation Schritte: - **Dependency Resolution** - **Source Code kompilieren** - **(Unit) Tests ausführen** - **Software packaging** - **In Repositories ausbringen** - **Dokumentation erstellen**

Tools: - Java: (Ant), **Maven**, Gradle. - C/C++: make. - Microsoft: MS Build, Visual Build. - Ruby: Rake.

Maven – Beispiel

- **POM** (Project Object Model): XML-basiert.
- Metadaten: Dependencies, Build Prozess, Dokumentation.
- POM kann **vererbt** werden (wie OO; „Super-POM“ von Maven).
- Aufruf: `mvn [options] [<goal(s)>] [<phase(s)>]`, z. B. `mvn clean package -DskipTests=true`.

GitHub Actions (CI)

- Stellt Infrastruktur, Requirements deklariert.
- (+) standardisiert, geringer Aufwand.
- (-) keine Kontrolle über Infrastruktur, Customization schwer.
- Konfiguration: Wann soll Build ausgeführt werden? Welche Schritte?

DevOps

Problematik – „Wall of Confusion”

- Entwicklungsteams arbeiten agil, Organisationen oft noch Wasserfall.
- Entwicklung **will Änderung** (Business, Dev, QA).
- Betrieb **will Stabilität** (Sysadmin, DBA, Network Engineer).
- Spannungsfeld:
 - Entwickler bringen nicht immer konsistente SW.
 - Entwicklungsprozess **agil**, Betriebsprozess **statisch**.
 - Betrieb widersteht Änderungen (Stabilität).
- → Verzögerungen → Verluste → **IT ist Flaschenhals** von Idee zum Markt.

DevOps – Lösung

- **Agile Development** überbrückt Business ↔ Dev/QA.
- **DevOps** überbrückt Dev/QA ↔ Operations.

DevOps Practices

- **Versionskontrolle für alle Bereiche** (auch Betrieb, z. B. Konfigurationsdateien).
- Automatisiertes Testen.
- **Proaktives Monitoring** (Metriken, auch wenn “alles passt”).
- Anwendung von **Kanban/Scrum**.
- CI/CD.
- „**Abrufbereitschaft**“ auch für Entwicklung.
- Virtualisierung/Container (**Docker, Kubernetes**).

***Wichtig:** “Even with the best tools, DevOps is just another buzzword if you don’t have the right culture.” → **Kulturwandel** notwendig.*

Architektur und Deployment

- **Traditionell (Monolith):** gesamte App auf eine Server-Instanz, oft mit CI.
 - **Microservices:** viele kleine Apps, viel mehr Deployments.
-

ZENTRALE PRÜFUNGSPUNKTE – Quick Reference

Mythen und Zitate

- **Fred Brooks:** „Adding manpower to a late software project makes it later” (Mythical Man-Month). „Wie verspätet sich ein Projekt um ein Jahr? Um einen Tag zu jedem Zeitpunkt.”
- **Dijkstra:** „Testing can show the presence of bugs, but not their absence.”
- **Balzer:** „Bei jeder SW-Entwicklung wird in der Regel das Rad mehrmals erfunden.”

Wichtige Namen

- **Royce:** Wasserfall (1970).
- **Boehm:** Spiral, COCOMO 81 (1981), COCOMO II (2000).
- **Albrecht (IBM):** Function Points (1979).
- **Nonaka & Takeuchi:** Scrum-Begriff.
- **Henry Gantt:** GANTT-Diagramm.
- **Kano:** Kundenzufriedenheitsmodell (1984).
- **Sommerville:** SE-Buch.
- **Chris Rupp:** Requirements Engineering.

Standards

- **ISO 9000/9001:** Qualitätsmanagement.
- **ISO/IEC 9126:1990 → 25010:2011:** Qualitätsfaktoren.
- **ISO/IEC/IEEE 29148-2011:** Qualitätskriterien Anforderungen.
- **IEEE 610.12:** Definitionen (QS, Fehler).
- **IEEE 828-1990:** SCM.
- **IEEE 830-1993:** Pflichtenheft-Inhalte.
- **IEEE 1061:** Metriken.
- **CMMI** (5 Stufen): Initial → Repeatable → Defined → Managed → Optimizing.
- **DIN 69901:** Projektmanagement.

Akronyme & Abkürzungen (alle merken)

- **RE:** Requirements Engineering
- **FA / NFA:** Funktionale / Nicht-funktionale Anforderungen
- **DoD:** Definition of Done
- **INVEST:** Independent, Negotiable, Valuable, Estimateable, Small, Testable
- **MVC / MVP:** Model-View-Controller / -Presenter
- **SOA:** Service Oriented Architecture
- **UDDI / WSDL / SOAP:** Web-Service-Standards

- **REST**: Representational State Transfer
- **DI / IoC**: Dependency Injection / Inversion of Control
- **AOP**: Aspect-Oriented Programming
- **CMMI**: Capability Maturity Model Integration
- **GQM**: Goal-Question-Metric
- **MTA / CTA**: Meilenstein-/Kosten-Trend-Analyse
- **WBS / PBS**: Work-/Project-Breakdown Structure
- **PERT / GANTT**: Planungsdiagramme
- **CI / CD**: Continuous Integration / Delivery / Deployment
- **SCM**: Software Configuration Management
- **PSP**: Projektstrukturplan
- **PRINCE2**: Projektmanagement-Methode
- **KDSI / KSLOC**: Kilo Delivered Source Instructions / Kilo Source Lines of Code
- **PM / CM**: Personenmonate / Kalendermonate (in COCOMO)
- **UFP / FP / VAF / TDI**: Function-Points-Berechnung
- **POM**: Project Object Model (Maven)
- **EI / EO / EQ / ILF / EIF**: Function-Points-Kategorien
- **CFD**: Cumulative Flow Diagram
- **PDCA**: Plan-Do-Check-Act (Qualitätsmgmt)
- **ROI**: Return on Investment
- **PoC**: Proof of Concept

Drei Wissensebenen nach Kano + Erhebungstechniken

Wissensebene	Faktoren	Techniken
bewusst	Leistungs-	Befragungstechniken (Interview, Fragebogen)
unbewusst	Begeisterungs-	Kreativitätstechniken (Brainstorming, 6-3-5, 6-Hut)
unterbewusst	Basis-	Beobachtungstechniken (Feldbeobachtung, Apprenticing) und artefaktbasiert (Hintergrundstudie, Systemarchäologie, Reuse)

Reviews-Tabelle

Review	Personen	Charakteristik
Stellungnahme	Autor + Prüfer	Markieren von Auffälligkeiten
Walkthrough	+ Moderator	Autor präsentiert + Gedankengänge
Inspektion	Inspektorengruppe	6 Phasen, formal, Checklisten

Magisches Dreieck im PM

Zeit – Kosten – Qualität – Funktionalität (eigentlich Viereck!)

MVC vs. MVP – Hauptunterschied

- MVC: View kann **direkt** auf Model zugreifen.
- MVP: View **passiv**, Presenter ist Mittelsmann → **bessere Testbarkeit**.

REST HTTP-Methoden

- GET = lesen
- POST = erstellen
- PUT = update/ersetzen
- DELETE = löschen

WSDL Hauptelemente

types, message, portType, binding

SOAP Aufbau

Envelope, (Header), Body, (Fault)

COCOMO Projekttypen + Faktoren

- Organic: 2.4 / 1.05 / 3.2
- Semidetached: 3.0 / 1.12 / 3.0
- Embedded: 3.6 / 1.20 / 2.8
- TDEV-Exponent: 0.38 / 0.35 / 0.32

Function Points Gewichtungen

- EI: 3 / 4 / 6
- EO: 4 / 5 / 7
- EQ: 3 / 4 / 6
- ILF: 7 / 10 / 15
- EIF: 5 / 7 / 10

Application Points Gewichtungen

- Screens: 1 / 2 / 3
- Reports: 2 / 5 / 8
- Module: $\times$ 10

Velocity & Story Points

- *13 Story Points = **Epic** (zu groß für einen Sprint)*
- Schätzung: Fibonacci-artige Größen (1, 2, 3, 5, 8, 13, 20, 40, 100)

Sprint-Längen und Team-Größen

- Sprint: ~ 1 Monat (+/- 1–2 Wochen)
- Daily Scrum: 15 Min, täglich, Stand-up
- Team: 5–10 Personen

Auftraggeber/Auftragnehmer

- **Lastenheft** = Auftraggeber, **WAS und WARUM** (Anforderungen einholen).
- **Pflichtenheft** = Auftragnehmer, **WIE** (Realisierungsvorgaben, technische Sprache).